

ZoSolvers

Zeroth-Order Optimisation Solvers with Random Oracles

User Manual — Version 0.1.5

Amir Ali Farzin

Contents

1	Introduction	3
2	Installation and Testing	4
2.1	Requirements	4
2.2	Install from PyPI (System-Wide)	4
2.3	Install from Wheel (System-Wide)	4
2.4	Install in a Poetry Environment	4
2.5	Development Install	4
2.6	Running the Demo Scripts	4
2.7	Running the Test Suite	5
3	Package Structure	6
4	Mathematical Background	7
4.1	Zeroth-Order Optimisation	7
4.2	Random Oracles	7
4.2.1	Gaussian Oracle	7
4.2.2	Sphere Oracle	8
4.2.3	Efficient Sampling	8
4.3	Minimax Oracles	8
4.3.1	Mini-Batching in the Minimax Setting	8
5	Modules	9
5.1	<code>utils.py</code>	9
5.1.1	<code>is_diagonal</code>	9
5.1.2	<code>resolve_n_jobs</code>	9
5.1.3	<code>chunk_sizes</code>	9
5.1.4	<code>ParallelBatchMixin</code>	9
5.2	<code>minimisation.py</code> — <code>Z0_gauss_min</code>	10
5.2.1	Constructor	10
5.2.2	Methods	11
5.3	<code>minimax.py</code> — <code>Z0_gauss_minmax</code>	12
5.3.1	Constructor	12
5.3.2	Methods	13
6	Solver Algorithms	15
6.1	Zeroth-Order Gradient Descent (ZOGD)	15
6.2	Zeroth-Order Extra-Gradient for Minimisation (ZOEgM)	15
6.3	Zeroth-Order Gradient Descent-Ascent (ZOGDA)	16
6.4	Zeroth-Order Extra-Gradient for Minimax (ZOEgmm)	16
6.5	Early Stopping Criteria	17
7	Parallel Function Evaluation	18

7.1	What Is Parallelised	18
7.2	Chunking	18
7.3	Backends	18
7.3.1	Randomness Under the Process Backend	19
7.4	The Worker Pool	19
7.5	When It Pays Off	19
8	Usage Examples	20
8.1	Minimisation	20
8.1.1	Basic Unconstrained Minimisation	20
8.1.2	Sphere Oracle	20
8.1.3	Precision Matrix B	20
8.1.4	Box-Constrained Minimisation	20
8.1.5	Parallel Mini-Batches	21
8.1.6	Growing Mini-Batch (<code>t = "iteration"</code>)	21
8.1.7	Early Stopping	21
8.1.8	Extra-Gradient	21
8.2	Minimax	21
8.2.1	Basic Unconstrained Minimax	21
8.2.2	Sphere Oracle with Independent Precision Matrices	22
8.2.3	Projected Minimax	22
8.2.4	Parallel Mini-Batches	22
8.2.5	Step-Size Ratio τ	22
8.2.6	Extra-Gradient Minimax	22
9	Testing	23
9.1	Unit and Statistical Tests	23
9.2	Demo Scripts	23
10	Parameter Guide	24

Chapter 1

Introduction

ZoSolvers is a Python package for zeroth-order (ZO) optimisation. It provides gradient-free solvers for both minimisation and minimax problems using random directional oracles. Two oracle families are supported:

- **Gaussian oracle:** the perturbation direction is sampled from a multivariate normal distribution.
- **Sphere oracle:** the perturbation direction is sampled uniformly from a (generalised) unit sphere.

Both oracle types can be equipped with a *precision matrix* B that shapes the distribution of perturbation directions, enabling preconditioning analogous to quasi-Newton methods.

Chapter 2

Installation and Testing

2.1 Requirements

- Python ≥ 3.10
- numpy $\geq 2.2.6$
- matplotlib $\geq 3.10.8$ (for the demo scripts)

Parallel evaluation (Chapter 7) uses only the Python standard library (`concurrent.futures`); it requires no additional dependency.

2.2 Install from [PyPI](#) (System-Wide)

```
pip install ZoSolvers
```

2.3 Install from Wheel (System-Wide)

After building the package with `poetry build`, install the generated wheel globally:

```
pip install dist/zosolvers-0.1.5-py3-none-any.whl
```

2.4 Install in a Poetry Environment

```
poetry install
```

2.5 Development Install

```
git clone https://github.com/amirali77farzin/ZoSolvers.git
cd ZoSolvers
poetry install
```

2.6 Running the Demo Scripts

```
poetry run python tests/demo_min.py           # minimisation demo
poetry run python tests/demo_minimax.py       # minimax demo
```

2.7 Running the Test Suite

```
poetry run pytest tests/ -v          # everything (208 tests)
poetry run pytest tests/min_test.py -v # minimisation only
poetry run pytest tests/minimax_test.py -v # minimax only
```

Chapter 3

Package Structure

```
ZoSolvers/
|-- pyproject.toml           package metadata and dependencies
|-- poetry.lock             pinned dependency versions
|-- .gitignore
|-- README.md
|-- Docs/
|   |-- ZoSolvers_Manual.tex   LaTeX source of this document
|   '-- ZoSolvers_Manual.pdf   this document
|
|-- src/
|   '-- ZoSolvers/
|       |-- __init__.py
|       |-- minimisation.py    ZO_gauss_min (minimisation solvers)
|       |-- minimax.py        ZO_gauss_minmax (minimax solvers)
|       '-- utils.py           shared utilities and parallel execution helpers
'-- tests/
    |-- conftest.py           pytest path setup
    |-- min_test.py           unit + statistical tests for minimisation
    |-- minimax_test.py       unit + statistical tests for minimax
    |-- demo_min.py           runnable minimisation demo with plots
    '-- demo_minimax.py       runnable minimax demo with plots
```

Chapter 4

Mathematical Background

4.1 Zeroth-Order Optimisation

In zeroth-order optimisation the gradient $\nabla f(x)$ is not available. Only *function evaluations* $f(x)$ can be queried. A *random oracle* approximates the gradient by finite differences along a random direction u :

Forward difference

$$\hat{g}(x) = \frac{f(x + \mu u) - f(x)}{\mu} \cdot c u,$$

Backward difference

$$\hat{g}(x) = \frac{f(x) - f(x - \mu u)}{\mu} \cdot c u,$$

Central (symmetric) difference

$$\hat{g}(x) = \frac{f(x + \mu u) - f(x - \mu u)}{2\mu} \cdot c u,$$

where $\mu > 0$ is the smoothing parameter and c, u depend on the oracle type (Section 4.2).

4.2 Random Oracles

4.2.1 Gaussian Oracle

Sample the direction $u \sim \mathcal{N}(0, B^{-1})$ where $B \succ 0$ is a user-supplied *precision matrix*. The oracle is

$$\hat{g}(x) = \frac{\Delta f}{\mu} \cdot B u, \quad \mathbb{E}[\hat{g}(x)] \approx \nabla f_\mu(x),$$

where f_μ is the Gaussian-smoothed objective. Unbiasedness follows from

$$\mathbb{E}[(Bu)(Bu)^\top] = B \mathbb{E}[uu^\top] B = B B^{-1} B = B, \quad \text{so} \quad B \mathbb{E}[uu^\top] \nabla f = \nabla f.$$

When $B = I$ the direction is standard Gaussian: $u \sim \mathcal{N}(0, I)$.

4.2.2 Sphere Oracle

Sample s uniformly on $\mathcal{S}^{d-1}$ (unit sphere) via normalising a standard Gaussian draw:

$$s = z / \|z\|, \quad z \sim \mathcal{N}(0, I).$$

Set $u = Ls$ where $L = \text{chol}(B^{-1})$ (Cholesky factor). The oracle is

$$\hat{g}(x) = \frac{\Delta f}{\mu} \cdot d \cdot Bu, \quad \mathbb{E}[\hat{g}(x)] \approx \nabla f_\mu(x).$$

The factor d (dimension) corrects for $\mathbb{E}[ss^\top] = \frac{1}{d}I$:

$$d \cdot B \mathbb{E}[uu^\top] \nabla f = d \cdot B \cdot L \frac{1}{d} I \cdot L^\top \nabla f = BB^{-1} \nabla f = \nabla f.$$

When $B = I$, $u = s$ is simply a uniform unit-sphere sample.

4.2.3 Efficient Sampling

To avoid recomputing matrix factorisations at every iteration, ZoSolvers caches the sampling structure at construction time:

B	Structure	Cached object
None	identity	—
diagonal	'diagonal'	$\text{diag}(B)^{-1/2}$
general	'full'	$L = \text{chol}(B^{-1})$

4.3 Minimax Oracles

For $\min_x \max_y f(x, y)$, ZoSolvers uses a simultaneous perturbation (SPSA-style) oracle. Directions u_x, u_y are drawn independently; a single finite-difference quotient

$$g = \frac{\Delta f(x + \mu u_x, y + \mu u_y)}{\mu \text{ or } 2\mu}$$

is computed. The partial oracles are then

$$\hat{g}_x = c_x \cdot g \cdot B_x u_x, \quad \hat{g}_y = c_y \cdot g \cdot B_y u_y,$$

with $c_x = d_x$ (sphere) or 1 (Gaussian), and similarly for c_y . Independence of u_x and u_y ensures cross-terms vanish and $\mathbb{E}[\hat{g}_x] \approx \partial_x f_\mu$, $\mathbb{E}[\hat{g}_y] \approx \partial_y f_\mu$.

Note that $\hat{g}_x$ and $\hat{g}_y$ share the same scalar quotient g , so a single finite difference — two function evaluations for a central difference — yields both partial oracles.

4.3.1 Mini-Batching in the Minimax Setting

A step with mini-batch size t draws t independent pairs $(u_x^{(i)}, u_y^{(i)})$, $i = 1, \dots, t$, and averages:

$$\hat{g}_x = \frac{1}{t} \sum_{i=1}^t c_x g^{(i)} B_x u_x^{(i)}, \quad \hat{g}_y = \frac{1}{t} \sum_{i=1}^t c_y g^{(i)} B_y u_y^{(i)}.$$

Within a single sample i the two partial oracles share the pair $(u_x^{(i)}, u_y^{(i)})$, which preserves the SPSA structure; across samples the pairs are independent, which is what makes the average reduce variance at the usual $O(1/\sqrt{t})$ rate.

Chapter 5

Modules

5.1 `utils.py`

5.1.1 `is_diagonal`

```
1 from ZoSolvers.utils import is_diagonal
2
3 is_diagonal(matrix, tol=1e-12)
```

Returns True if `matrix` is square and diagonal (up to tolerance `tol`). Raises `RuntimeError` for non-square input. Used internally to select the efficient sampling path.

5.1.2 `resolve_n_jobs`

```
1 from ZoSolvers.utils import resolve_n_jobs
2
3 resolve_n_jobs(n_jobs)
```

Maps an `n_jobs` argument onto a concrete positive worker count. 1 (or `None`) means sequential execution; -1 means every CPU core, -2 all but one, and so on. Raises `ValueError` for 0, or for a negative value exceeding the core count.

5.1.3 `chunk_sizes`

```
1 from ZoSolvers.utils import chunk_sizes
2
3 chunk_sizes(total, n_chunks)
```

Splits `total` samples into at most `n_chunks` balanced, strictly positive chunks (sizes differ by at most one). When `n_chunks` exceeds `total`, only `total` chunks are produced, so no worker is handed an empty batch.

5.1.4 `ParallelBatchMixin`

The base class both solvers inherit from. It owns the worker pool: lazy creation on first use, reuse across every iteration, and release through `close()` or the context-manager protocol. See Chapter 7.

5.2 minimisation.py — ZO_gauss_min

5.2.1 Constructor

```

1 from ZoSolvers.minimisation import ZO_gauss_min
2
3 opt = ZO_gauss_min(
4     func,                # callable: f(x) -> float
5     x0,                  # array_like: initial point
6     h = 1e-3,            # float: step size
7     mu = 1e-5,           # float: smoothing parameter
8     N = 10000,           # int: max iterations
9     t = 1,               # int / "iteration": samples per oracle
10    call
11    tol_f = None,         # float / None: relative function tolerance
12    tol_g = None,         # float / None: oracle-norm tolerance
13    B = None,             # ndarray / None: precision matrix
14    proj = None,          # callable / None: projection onto feasible
15    set
16    oracle_type = "gaussian", # "gaussian" / "sphere"
17    project_init = False,   # bool: also project x0 itself
18    n_jobs = 1,            # int: workers for the t samples (-1 = all
19    cores)
20    backend = "thread"     # "thread" / "process"
21 )

```

Parameters

Parameter	Description
<code>func</code>	Objective function $f : \mathbb{R}^d \rightarrow \mathbb{R}$. Must accept a NumPy array.
<code>x0</code>	Starting point. Converted to a float array internally.
<code>h</code>	Step size. In ZOGD this is the gradient step; in ZOEGm it is the first-stride step.
<code>mu</code>	Smoothing parameter $\mu > 0$.
<code>N</code>	Maximum number of iterations.
<code>t</code>	Number of oracle samples averaged per step. Setting <code>"iteration"</code> uses the current iteration number k as the mini-batch size (increasing schedule).
<code>tol_f</code>	Stop early when $f(x_k) \leq \text{tol_f} \cdot f(x_0)$.
<code>tol_g</code>	Stop early when the empirical squared oracle norm $\leq \text{tol_g}$.
<code>B</code>	Precision matrix $B \succ 0$ (inverse of the covariance of the perturbation distribution). <code>None</code> $\Rightarrow$ identity.
<code>proj</code>	Projection function $\text{proj}_{\mathcal{X}} : \mathbb{R}^d \rightarrow \mathcal{X}$. <code>None</code> $\Rightarrow$ unconstrained.
<code>oracle_type</code>	<code>"gaussian"</code> : $u \sim \mathcal{N}(0, B^{-1})$. <code>"sphere"</code> : u uniform on B -metric unit sphere.
<code>project_init</code>	Whether x_0 itself is projected onto $\mathcal{X}$ before the first step. <code>False</code> (default) returns x_0 exactly as supplied, so the first row of the trajectory may be infeasible if $x_0 \notin \mathcal{X}$; <code>True</code> makes every returned iterate feasible. No effect when <code>proj</code> is <code>None</code> .
<code>n_jobs</code>	Number of workers used to evaluate the t samples of one step. <code>1</code> (default) is sequential; <code>-1</code> uses all cores, <code>-2</code> all but one. See Chapter 7.
<code>backend</code>	<code>"thread"</code> (default) or <code>"process"</code> .

5.2.2 Methods

`oracle(x, method)`

Returns a single zeroth-order gradient estimate at x using the specified finite-difference `method` (`"forw"`, `"back"`, or `"center"`).

`batch_oracle(x, method, n)`

Returns the mean of `n` independent oracle draws at x . This is the mini-batch the solver takes a step along, and the unit of work that is split across `n_jobs` workers.

`close()` and the context-manager protocol

Shuts down the worker pool. Safe to call repeatedly, and unnecessary when `n_jobs = 1` (no pool is ever created). Using the solver in a `with` block calls it automatically.

ZOGD(method="forw")

Runs Zeroth-Order Gradient Descent. Returns a NumPy array of shape (K, d) where $K \leq N$ is the number of iterates (including x_0). Row 0 is x_0 as supplied unless `project_init=True`.

ZOEGm(method="forw", gamma=1.0)

Runs the Zeroth-Order Extra-Gradient method for minimisation. The parameter `gamma` scales the second stride. Returns an array of shape (K, d) .

5.3 minimax.py — ZO_gauss_minmax

5.3.1 Constructor

```

1 from ZoSolvers.minimax import ZO_gauss_minmax
2
3 opt = ZO_gauss_minmax(
4     func,                # callable: f(x, y) -> float
5     x0,                  # array_like: initial minimiser point
6     y0,                  # array_like: initial maximiser point
7     h = 1e-3,            # float: base step size (y-direction)
8     tau = 1,             # float: step-size ratio (x-step = h / tau)
9     mu = 1e-5,           # float: smoothing parameter
10    N = 10000,            # int: max iterations
11    t = 1,                # int / "iteration": samples per oracle
12    call,
13    tol = None,           # float / None: projected-operator-norm
14    tolerance,
15    B_x = None,           # ndarray / None: precision matrix for x
16    B_y = None,           # ndarray / None: precision matrix for y
17    proj_x = None,        # callable / None: projection for x
18    proj_y = None,        # callable / None: projection for y
19    oracle_type = "gaussian", # "gaussian" / "sphere"
20    project_init = False,  # bool: also project x0 and y0 themselves
21    n_jobs = 1,           # int: workers for the t samples (-1 = all
22    cores),
23    backend = "thread"    # "thread" / "process"
24 )

```

Parameters

Parameter	Description
<code>func</code>	Saddle-point objective $f : \mathbb{R}^{d_x} \times \mathbb{R}^{d_y} \rightarrow \mathbb{R}$, minimised in x , maximised in y .
<code>x0</code> , <code>y0</code>	Starting points for minimiser and maximiser.
<code>h</code>	Base step size. The y -update uses step h ; the x -update uses step h/τ .
<code>tau</code>	Step-size ratio $\tau > 0$. Setting $\tau > 1$ makes the x -step smaller than the y -step.
<code>mu</code>	Smoothing parameter $\mu > 0$.
<code>N</code>	Maximum number of iterations.
<code>t</code>	Samples per oracle call, or "iteration" for a growing schedule.
<code>tol</code>	Stop early when the empirical squared projected-operator norm $\leq \text{tol}$.
<code>B_x</code> , <code>B_y</code>	Independent precision matrices for x and y directions.
<code>proj_x</code> , <code>proj_y</code>	Independent projection functions. <code>None</code> $\Rightarrow$ unconstrained.
<code>oracle_type</code>	"gaussian" or "sphere" (applied to both x and y).
<code>project_init</code>	Whether x_0 and y_0 are projected onto $\mathcal{X}$ and $\mathcal{Y}$ before the first step. <code>False</code> by default. Each side is handled by its own projection, so with only <code>proj_x</code> supplied just x_0 is moved.
<code>n_jobs</code>	Number of workers used to evaluate the t samples of one step. 1 (default) is sequential; -1 uses all cores. See Chapter 7.
<code>backend</code>	"thread" (default) or "process".

5.3.2 Methods

`oracle(x, y, ux, uy, xy, method)`

Returns the partial zeroth-order oracle for variable `xy` ('`x`' or '`y`') using pre-sampled directions `ux`, `uy`, under the SPSA simultaneous perturbation.

Because both partial oracles are built from the same scalar quotient, the solver internally uses the paired form, which evaluates f once and returns $(\hat{g}_x, \hat{g}_y)$ together. Calling `oracle` twice with the same `ux`, `uy` gives identical values at twice the number of function evaluations.

`batch_oracle(x, y, method, n)`

Returns $(\bar{g}_x, \bar{g}_y)$, the mean partial oracles over `n` independent direction pairs (Section 4.3.1). This is the unit of work split across `n_jobs` workers.

`close()` and the context-manager protocol

Shuts down the worker pool; see the minimisation counterpart above.

`ZOGDA(method="forw")`

Zeroth-Order Gradient Descent-Ascent. Returns `(x_traj, y_traj)`, each a NumPy array of shape (K, d_x) and (K, d_y) respectively.

`ZOEGmm(method="forw", gamma=1.0)`

Zeroth-Order Extra-Gradient for minimax. Two independent mini-batches are drawn per iteration, one for the look-ahead stride and one for the update stride. Returns `(x_traj, y_traj)`.

Chapter 6

Solver Algorithms

6.1 Zeroth-Order Gradient Descent (ZOGD)

Algorithm 1 ZOGD — Zeroth-Order Gradient Descent

Input: f , x_0 , step size h , smoothing μ , iterations N , samples t , projection $\text{proj}_{\mathcal{X}}$, oracle type**Output:** Trajectory $\{x_k\}_{k=0}^K$ 1: **if** `project_init` **then** $x_0 \leftarrow \text{proj}_{\mathcal{X}}(x_0)$ 2: **for** $k = 0, 1, \dots, N - 1$ **do**3: Draw t directions $u^{(1)}, \dots, u^{(t)}$ from the oracle4: $\hat{g}_k \leftarrow \frac{1}{t} \sum_{i=1}^t \hat{g}(x_k, u^{(i)}, \mu)$ ▷ evaluated across `n_jobs` workers5: $x_{k+1} \leftarrow \text{proj}_{\mathcal{X}}(x_k - h \hat{g}_k)$ 6: **check** early-stopping criteria7: **end for**

The t summands are independent, so line 3 is the step that Chapter 7 parallelises. Every iterate the loop produces is feasible; the returned trajectory additionally contains x_0 , which is projected only when `project_init=True`.

6.2 Zeroth-Order Extra-Gradient for Minimisation (ZOEgM)

The extra-gradient method takes two oracle-based steps per iteration (a *look-ahead* step and an *update* step), which improves convergence on certain problem classes.

Algorithm 2 ZOEgM — Zeroth-Order Extra-Gradient (Minimisation)

Input: f , x_0 , h , γ , μ , N , t , $\text{proj}_{\mathcal{X}}$, oracle type**Output:** Trajectory $\{x_k\}_{k=0}^K$ 1: **for** $k = 0, 1, \dots, N - 1$ **do**2: // *Look-ahead step*3: $\hat{x}_k \leftarrow \text{proj}_{\mathcal{X}}(x_k - h \hat{g}(x_k))$ 4: // *Update step, taken from the projected look-ahead point*5: $x_{k+1} \leftarrow \text{proj}_{\mathcal{X}}(\hat{x}_k - \gamma h \hat{g}(\hat{x}_k))$ 6: **check** early-stopping criteria7: **end for**

Both $\hat{g}$ evaluations are mini-batches of t samples, each parallelised as in ZOGD.

6.3 Zeroth-Order Gradient Descent-Ascent (ZOGDA)

For saddle-point problems $\min_x \max_y f(x, y)$, ZOGDA simultaneously updates both variables using their respective partial oracles.

Algorithm 3 ZOGDA — Zeroth-Order Gradient Descent-Ascent

Input: $f, x_0, y_0, h, \tau, \mu, N, t, \text{proj}_{\mathcal{X}}, \text{proj}_{\mathcal{Y}}, \text{oracle type}$

Output: Trajectories $\{x_k\}, \{y_k\}$

```

1: if project_init then  $x_0 \leftarrow \text{proj}_{\mathcal{X}}(x_0), y_0 \leftarrow \text{proj}_{\mathcal{Y}}(y_0)$ 
2: for  $k = 0, 1, \dots, N - 1$  do
3:   for  $i = 1, \dots, t$  do ▷ across n_jobs workers
4:     Draw  $u_x^{(i)}, u_y^{(i)}$  independently from the oracle
5:      $g^{(i)} \leftarrow$  finite difference of  $f$  along  $(u_x^{(i)}, u_y^{(i)})$  at  $(x_k, y_k)$ 
6:      $\hat{g}_x^{(i)} \leftarrow c_x g^{(i)} B_x u_x^{(i)}, \hat{g}_y^{(i)} \leftarrow c_y g^{(i)} B_y u_y^{(i)}$ 
7:   end for
8:    $\hat{g}_x \leftarrow \frac{1}{t} \sum_{i=1}^t \hat{g}_x^{(i)}, \hat{g}_y \leftarrow \frac{1}{t} \sum_{i=1}^t \hat{g}_y^{(i)}$ 
9:    $x_{k+1} \leftarrow \text{proj}_{\mathcal{X}}(x_k - \frac{h}{\tau} \hat{g}_x)$ 
10:   $y_{k+1} \leftarrow \text{proj}_{\mathcal{Y}}(y_k + h \hat{g}_y)$ 
11:  check early-stopping criterion
12: end for
```

Each sample costs one finite difference — two evaluations of f for a central difference — and that single quotient $g^{(i)}$ produces both partial oracles.

6.4 Zeroth-Order Extra-Gradient for Minimax (ZOEgmm)

ZOEgmm extends the extra-gradient idea to the minimax setting. Two independent mini-batches are drawn per iteration: one for the look-ahead step and one for the update step¹.

¹More details can be found in “Farzin, A.A., Pun, Y.M., Lesage-Landry, A., Diouane, Y. and Shames, I., 2025. Min-max optimisation for nonconvex-nonconcave functions using a random zeroth-order extragradient algorithm. Transactions on Machine Learning Research.”

Algorithm 4 ZOEGmm — Zeroth-Order Extra-Gradient (Minimax)**Input:** $f, x_0, y_0, h, \tau, \gamma, \mu, N, t, \text{proj}_{\mathcal{X}}, \text{proj}_{\mathcal{Y}}, \text{oracle type}$ **Output:** Trajectories $\{x_k\}, \{y_k\}$

```

1: if project_init then  $x_0 \leftarrow \text{proj}_{\mathcal{X}}(x_0), y_0 \leftarrow \text{proj}_{\mathcal{Y}}(y_0)$ 
2: for  $k = 0, 1, \dots, N - 1$  do
3:   Draw a mini-batch of  $t$  independent pairs and average it into  $(\hat{g}_x^{(1)}, \hat{g}_y^{(1)})$ 
4:   // Look-ahead step
5:    $\hat{x}_k \leftarrow \text{proj}_{\mathcal{X}}\left(x_k - \frac{h}{\tau} \hat{g}_x^{(1)}\right)$ 
6:    $\hat{y}_k \leftarrow \text{proj}_{\mathcal{Y}}\left(y_k + h \hat{g}_y^{(1)}\right)$ 
7:   Draw a second, independent mini-batch of  $t$  pairs and average it into  $(\hat{g}_x^{(2)}, \hat{g}_y^{(2)})$ 
8:   // Update step, taken from the projected look-ahead point
9:    $x_{k+1} \leftarrow \text{proj}_{\mathcal{X}}\left(\hat{x}_k - \frac{\gamma h}{\tau} \hat{g}_x^{(2)}\right)$ 
10:   $y_{k+1} \leftarrow \text{proj}_{\mathcal{Y}}\left(\hat{y}_k + \gamma h \hat{g}_y^{(2)}\right)$ 
11:  check early-stopping criterion
12: end for

```

6.5 Early Stopping Criteria

Minimisation (tol_f) The algorithm stops when

$$f(x_k) \leq \text{tol_f} \cdot f(x_0).$$

Minimisation (tol_g) The algorithm stops when

$$\frac{1}{10} \sum_{j=1}^{10} \|\hat{g}(x_k)\|^2 \leq \text{tol_g}.$$

For projected problems, the oracle norm is replaced by the squared projected-residual proxy $\|(x_k - \text{proj}_{\mathcal{X}}(x_k - h \hat{g}))/h\|^2$.

Minimax (tol) The algorithm stops when the empirical squared projected-operator norm

$$\frac{1}{10} \sum_{j=1}^{10} \left(\left\| \frac{x_k - \text{proj}_{\mathcal{X}}(x_k - \frac{h}{\tau} \hat{g}_x)}{h/\tau} \right\|^2 + \left\| \frac{\text{proj}_{\mathcal{Y}}(y_k + h \hat{g}_y) - y_k}{h} \right\|^2 \right) \leq \text{tol}.$$

Cost of the criteria Each criterion is checked only on a fixed schedule (every $N/50$ iterations for **tol_f** and **tol_g**, every $N/100$ for the minimax **tol**, and at least once per iteration for small N), so its ten probe samples add little to the total budget. Those probes are themselves spread over the **n_jobs** workers.

Chapter 7

Parallel Function Evaluation

7.1 What Is Parallelised

Every solver step averages t oracle samples. Those samples are independent draws, so nothing forces them to be evaluated one after another. Passing `n_jobs > 1` spreads them over a pool of workers:

```
1 opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=32, n_jobs=-1)
2 x_traj = opt.ZOGD(method="center")
3 opt.close()
```

`n_jobs=-1` requests every core, `-2` all but one. The default `n_jobs=1` runs sequentially and never creates a pool, so existing code is unaffected.

What is *not* parallelised is the iteration loop itself: x_{k+1} depends on x_k , so steps are inherently sequential. The speed-up therefore comes from within a step and is bounded by t .

7.2 Chunking

Samples are grouped into one chunk per worker rather than dispatched one task per sample. A batch of t samples over w workers becomes $\min(t, w)$ tasks of $\lceil t/w \rceil$ or $\lfloor t/w \rfloor$ samples each, so the scheduling overhead is paid w times per step instead of t times. When $w > t$, only t chunks are created and no worker receives an empty batch.

7.3 Backends

backend	When to use it
"thread"	Default. Accepts any callable, including lambdas, closures and bound methods, and shares memory with the caller. Delivers a real speed-up whenever <code>func</code> releases the GIL — NumPy-heavy models, compiled extensions, and subprocess- or network-backed simulators, which covers most zeroth-order applications.
"process"	True parallelism for a <code>func</code> that is pure Python and CPU-bound. <code>func</code> must be picklable, so it has to be a module-level function rather than a lambda.

7.3.1 Randomness Under the Process Backend

Forked workers inherit the parent's NumPy random state, which would make every worker draw the same perturbations. Each worker process therefore reseeds itself from the operating system entropy pool when it starts. The practical consequence is that `np.random.seed` in the parent no longer makes a run reproducible under `backend="process"`. The thread backend shares the parent's generator and is unaffected.

7.4 The Worker Pool

The pool is created lazily on the first mini-batch and reused for every iteration afterwards; it is never rebuilt per step. Release it with `close()`, or let the context manager do it:

```
1 with ZO_gauss_minmax(f, x0, y0, h=1e-3, mu=1e-8, N=10000,
2                       t=16, n_jobs=8) as opt:
3     x_traj, y_traj = opt.ZOGDA(method="center")
4 # pool shut down here
```

7.5 When It Pays Off

Parallelism helps when one evaluation of `func` is expensive relative to the cost of handing work to a worker. For a cheap analytic objective the overhead dominates and `n_jobs=1` is faster. The table below is from `tests/demo_min.py` on a 12-core machine, using an objective made to cost about 2 ms per call to stand in for a simulator ($N = 30$, $t = 8$, i.e. 480 evaluations per run):

n_jobs	Wall time	Speed-up
1	1.07 s	1.00×
2	0.55 s	1.93×
4	0.30 s	3.60×
-1 (12 cores)	0.16 s	6.56×

Scaling is close to linear until t or the core count runs out. Both demo scripts print this comparison when run.

Chapter 8

Usage Examples

8.1 Minimisation

8.1.1 Basic Unconstrained Minimisation

```
1 import numpy as np
2 from ZoSolvers.minimisation import ZO_gauss_min
3
4 def f(x):
5     return x[0]**2 + x[1]**2 + x[0]*x[1]
6
7 x0 = np.array([5.0, -5.0])
8 opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10)
9
10 x_traj = opt.ZOGD(method="center")
11
12 print("Final iterate:", x_traj[-1])
13 print("Final f value:", f(x_traj[-1]))
```

8.1.2 Sphere Oracle

```
1 opt_sph = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10,
2                       oracle_type="sphere")
3 x_traj = opt_sph.ZOGD(method="center")
```

8.1.3 Precision Matrix B

```
1 B_diag = np.diag([10.0, 0.01])
2 opt_diag = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10, B=B_diag)
3
4 B_full = np.array([[20.0, 0.01],
5                   [0.01, 0.02]])
6 opt_full = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10, B=B_full)
```

8.1.4 Box-Constrained Minimisation

```
1 proj = lambda x: np.clip(x, -2.0, 2.0) # project onto [-2, 2]^2
2
3 opt_proj = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10, proj=proj)
4 x_traj = opt_proj.ZOGD(method="center")
```

Every iterate the solver computes is feasible, but row 0 of the returned trajectory is the x_0 that was passed in, returned untouched. With $x_0 = [5, -5]$ and the box $[-2, 2]^2$ above, that first row is infeasible. Set `project_init=True` to project it as well:

```
1 opt_proj = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10, proj=proj,
2                       project_init=True)
3 x_traj = opt_proj.ZOGD(method="center")
4
5 print(x_traj[0])
```

8.1.5 Parallel Mini-Batches

```
1 # Expensive objective: a simulator, a NumPy-heavy model, an external
   solver
2 opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=32, n_jobs=-1)
3 x_traj = opt.ZOGD(method="center")
4 opt.close()
5
6 # Equivalent, with automatic clean-up
7 with ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=32, n_jobs=-1) as
   opt:
8     x_traj = opt.ZOGD(method="center")
9
10 # Pure-Python CPU-bound objective: f must be a module-level function
11 with ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=32,
12                  n_jobs=4, backend="process") as opt:
13     x_traj = opt.ZOGD(method="center")
```

8.1.6 Growing Mini-Batch (t = "iteration")

```
1 opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=500, t="iteration")
2 x_traj = opt.ZOGD(method="center")
3 # At iteration k, k oracle samples are averaged.
```

8.1.7 Early Stopping

```
1 # Stop when f(x) drops to 1% of f(x0)
2 opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=5000, t=10, tol_f=0.01)
3 x_traj = opt.ZOGD(method="center")
4 print(f"Stopped after {len(x_traj)} iterations.")
```

8.1.8 Extra-Gradient

```
1 opt = ZO_gauss_min(f, x0, h=1e-2, mu=1e-5, N=2000, t=10)
2 x_traj = opt.ZOEGm(method="center", gamma=1.0)
```

8.2 Minimax

8.2.1 Basic Unconstrained Minimax

```
1 from ZoSolvers.minimax import ZO_gauss_minimax
2
3 def f(x, y):
4     return ( x[0]**2 + x[1]**2 + x[0]*x[1]
```

```

5         - y[0]**2 - y[1]**2 + 1.5*y[0]*y[1] + 3*y[1])
6
7 x0 = np.array([10.0, -10.0])
8 y0 = np.array([ 9.0, -9.0])
9
10 opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5)
11 x_traj, y_traj = opt.ZOGDA(method="center")
12
13 print("Final x:", x_traj[-1])
14 print("Final y:", y_traj[-1])

```

8.2.2 Sphere Oracle with Independent Precision Matrices

```

1 B_x = np.diag([2.0, 0.2])
2 B_y = np.array([[2.0, 0.01],
3                 [0.01, 0.2]])
4
5 opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5,
6                       B_x=B_x, B_y=B_y, oracle_type="sphere")
7 x_traj, y_traj = opt.ZOGDA(method="center")

```

8.2.3 Projected Minimax

```

1 proj = lambda v: np.clip(v, -7.0, 7.0)
2
3 opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5,
4                       proj_x=proj, proj_y=proj)
5 x_traj, y_traj = opt.ZOGDA(method="center")
6
7 opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5,
8                       proj_x=proj, proj_y=proj, project_init=True)
9 x_traj, y_traj = opt.ZOGDA(method="center")

```

8.2.4 Parallel Mini-Batches

```

1 with ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000,
2                       t=16, n_jobs=-1) as opt:
3     x_traj, y_traj = opt.ZOEGmm(method="center", gamma=0.8)

```

8.2.5 Step-Size Ratio τ

```

1 opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=2, mu=1e-8, N=10000, t=5)
2 x_traj, y_traj = opt.ZOGDA(method="center")

```

8.2.6 Extra-Gradient Minimax

```

1 opt = ZO_gauss_minmax(f, x0, y0, h=1e-3, tau=1, mu=1e-8, N=10000, t=5)
2 x_traj, y_traj = opt.ZOEGmm(method="center", gamma=0.8)

```

Chapter 9

Testing

The `tests/` directory contains two test files and two demo scripts.

9.1 Unit and Statistical Tests

`min_test.py` Tests for `Z0_gauss_min`. Coverage: initialisation attributes and error handling; Cholesky cache correctness; sampling shapes and dispatch; oracle unbiasedness for Gaussian and sphere under identity, diagonal, and full B (statistical tests with 20 000 samples); ZOGD and ZOEGm convergence on a quadratic for all oracle/ B combinations; projection feasibility and `project_init`; early stopping; `t="iteration"` mode; small- N safety (no `ZeroDivisionError`); mini-batch mean and variance reduction; and the parallel surface, namely `n_jobs` resolution, chunk partitioning, pool lifecycle, and both backends.

`minimax_test.py` Tests for `Z0_gauss_minimax`. Same coverage adapted for the minimax setting, plus: independent B_x/B_y caches; partial oracle unbiasedness for both x and y ; bilinear cross-term cancellation; `proj_x/proj_y` non-mutation test; and regression tests for the 0.1.5 mini-batch fix — that the t draws of a batch are independent, that the error decreases with t , and that a central-difference sample costs two function evaluations rather than four.

The suite contains 208 tests.

Run all tests:

```
poetry run pytest tests/ -v
```

9.2 Demo Scripts

`demo_min.py` Runs nine experiments covering oracle types, finite-difference methods, B matrices, projection and `project_init`, mini-batch schedules, early stopping, and a timed `n_jobs` comparison. Prints a tabular summary and displays a 2×3 figure of convergence curves.

`demo_minimax.py` Equivalent demonstrations for the minimax solvers, additionally covering the step-size ratio τ and independent B_x, B_y configurations, and likewise ending with a timed `n_jobs` comparison.

Chapter 10

Parameter Guide

Parameter	Effect	Typical range
<code>h</code>	Controls descent speed. Too large $\Rightarrow$ divergence; too small $\Rightarrow$ slow.	$10^{-8} - 10^{-1}$
<code>mu</code>	Finite-difference step. Smaller reduces bias but increases variance.	$10^{-8} - 10^{-3}$
<code>t</code>	Larger t reduces oracle variance but costs more function evaluations per step.	1 – 50
<code>B</code>	Shapes the perturbation distribution. A good B approximates the Hessian.	Problem-dependent
<code>tau</code>	(Minimax) Balances x and y convergence rates.	0.5 – 5
<code>gamma</code>	(Extra-gradient) Second-stride scale. Smaller $\Rightarrow$ more conservative update.	0.5 – 2
<code>oracle_type</code>	<code>sphere</code> has lower variance per sample; <code>gaussian</code> is simpler and often equally effective.	" <code>gaussian</code> "
<code>n_jobs</code>	Workers evaluating the t samples of a step. Worth raising only when one call to <code>func</code> is expensive; the gain is capped by t .	1, or -1 for all cores
<code>backend</code>	" <code>thread</code> " suits GIL-releasing objectives (simulators, NumPy, compiled code); " <code>process</code> " suits pure-Python CPU-bound ones.	" <code>thread</code> "
<code>project_init</code>	Whether the initial guess is projected onto the feasible set before the first step.	<code>False</code>